

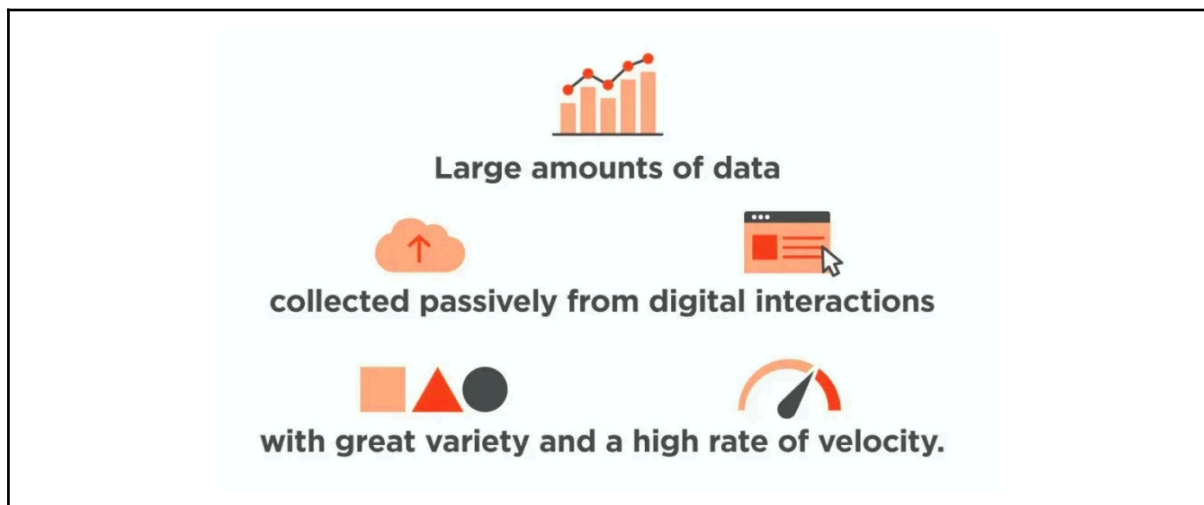
Section 7: Storage in System Design

Big Data Fundamentals

In this lecture we will explore fundamentals of big data, what makes data big, why traditional systems struggle at scale, and the architectural principles used to store and process massive datasets efficiently. So let's get started.

What is Big Data?

Almost every modern system today generates data continuously. Every user click, API request, sensor reading, sensor reading, log entry, video upload, or mobile interaction creates data that needs to be stored and analyzed. The challenge is that the volume of this data has grown far beyond what traditional databases and single machine systems were originally designed to handle. At a certain scale, the problem is no longer just storing the data, it is processing it efficiently, extracting insights from it, and doing so in a cost-effective manner. That is where the concept of Big Data comes in.

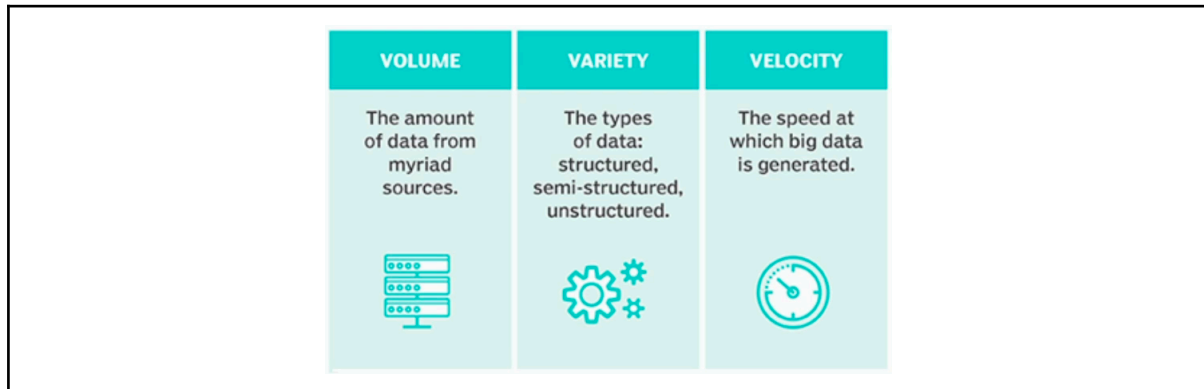


Big data refers to datasets that are so large, fast-moving, and complex that conventional data processing approaches become inadequate. This shift was driven by the rise of web platforms, mobile applications, IoT devices, machine-generated logs, and concerned systems producing data around the clock. To handle this new reality, the industry moved towards distributed storage, parallel processing, and specialized analytics frameworks.

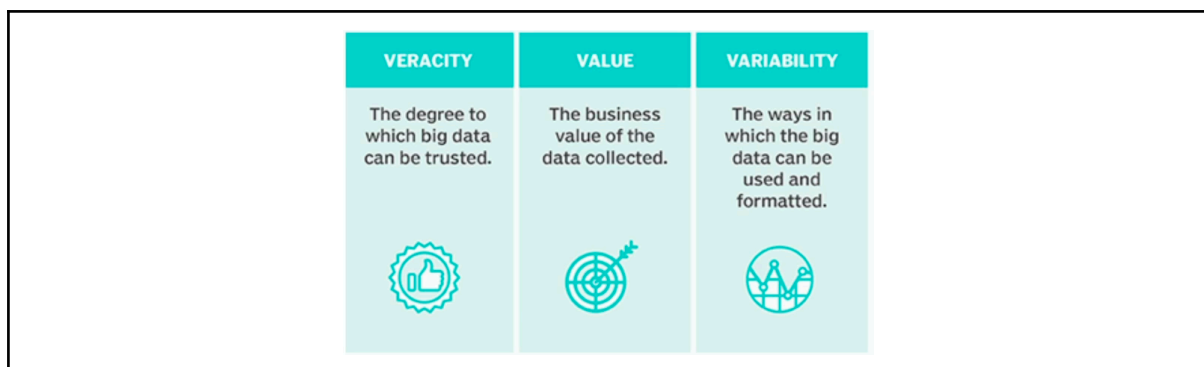
So in this remaining lecture, we will explore the characteristics of big data and understand why modern architectures had to evolve to support this.

The 6 V's of Big Data

When engineers talk about big data, they are usually not referring to the size alone. The real challenge comes from multiple dimensions of complexity, which is why the industry adopted the 6V framework.



1. **Volume** – The most visible dimension, referring to the sheer scale of data being generated. However, storing large amounts of data is only part of the problem.
2. **Velocity** – Adds another layer of complexity because data often arrives continuously and needs to be processed in near real-time. Examples include stock trades, user activity streams, or IoT telemetry, where delayed processing can reduce business value.
3. **Variety** – Modern systems rarely deal with a single data format. A single application might combine transactional records, JSON events, images, videos, and log files. This diversity forces architects to move beyond traditional relational database models and adopt more flexible storage and processing approaches.



4. **Veracity** – Just because data exists doesn't mean it is reliable, which is why veracity comes into the picture. It focuses on data quality — missing fields, duplicates, noisy sensor readings, and inconsistent records can easily produce misleading results. In many real-world projects, cleaning data is often harder than analyzing the data.
5. **Value** – Organisations invest in big data for value. The goal is not to collect more data, but to generate better insights, improve decisions, and create measurable business outcomes. Without value, even petabytes of data are just an expensive liability.

6. **Variability** – Reminds us that data is rarely static. User behaviour changes, languages evolve, traffic patterns fluctuate, and data meaning can shift depending on context. Systems must be designed to adapt to these changes rather than assuming predictable inputs.

As architects, these six dimensions provide a useful lens for evaluating a useful lens for evaluating data systems. They help us understand why big data requires specialised storage, processing, and analytics technologies instead of traditional approaches.

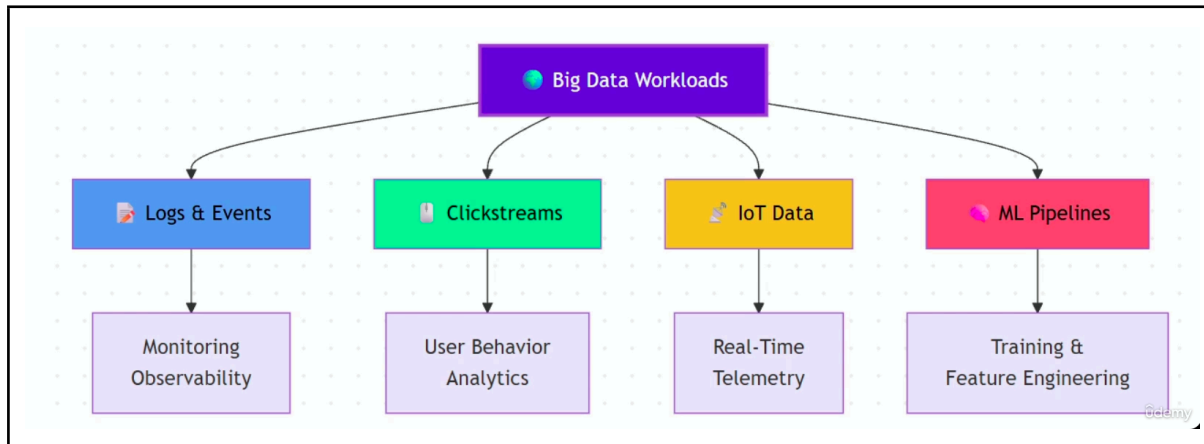
Why Traditional Storage Fails at Scale

Understanding why traditional storage fails is important because it explains why the entire big data ecosystem had to evolve in the first place. For many years scaling storage was straight forward. You bought a bigger server, added more disk, more memory, and more processing power. That approach works up to a point, but modern systems generate data at a scale where a single machine eventually becomes both a technical and economic bottleneck. Below are some of the challenges eventually faced by traditional systems:

- **Limited Scalability:** When datasets grow from gigabytes to terabytes and eventually petabytes, continuously upgrading one server is neither practical nor sustainable. You need the ability to spread data across multiple machines instead of relying on one.
- **Performance Bottleneck:** Traditional storage architectures were not designed for thousands of concurrent reads and writes happening across large distributed workloads. As demands increase, contention grows, latency rises, and throughput becomes a constraint.
- **Cost Inefficiencies:** High-end enterprise software can be extremely expensive, and every upgrade becomes larger and more costlier than the last one. This creates diminishing returns as the system grows.
- **Lack of Fault Tolerance:** In large-scale environments, hardware failures are not exceptional events. They are expected events. Traditional storage often treats failures as emergencies, whereas modern distributions are designed with replication and automatic recovery built in from the start itself.

This is why big data platforms moved toward distributed storage architectures that scale horizontally, handle failures gracefully, and deliver performance by leveraging many machines working together rather than one increasingly powerful server only.

Common Big Data Workloads



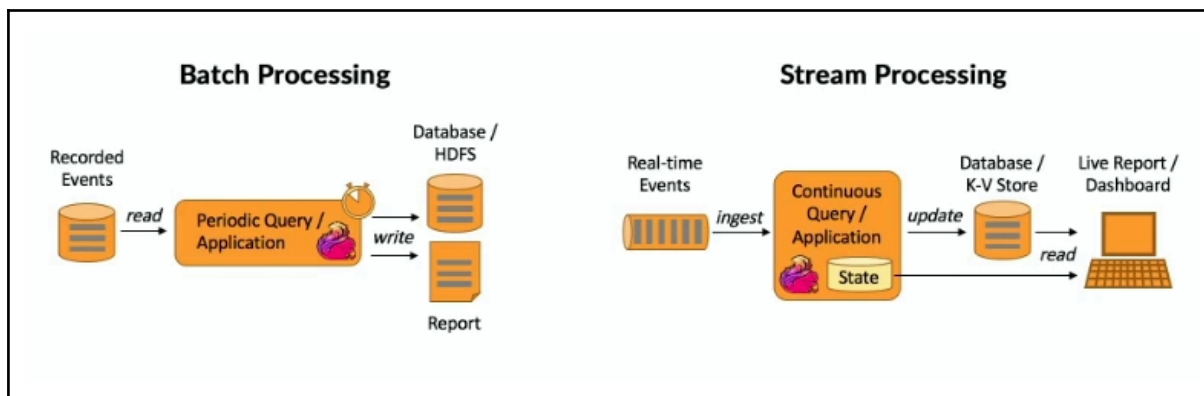
Big data isn't just about storing large amounts of information. It exists because modern applications generate data continuously as part of their day-to-day operation. Understanding these workloads helps us understand the architectural requirements behind big data platforms.

- **Logs & Events:** Every service, API, database, and server generates operational data that teams use for monitoring, troubleshooting, security analysis, and observability. At scale, even a moderately-sized platform can produce millions of log events every day.
- **Clickstreams:** Every page view, search, click and user interaction creates a digital trail. Organizations analyze this data to understand user behaviour, improve customer experience, optimize conversations, and drive recommendation engines.
- **IoT Data:** Sensors, smart devices, vehicles, and industrial equipment all generate continuous streams of telemetry data. The value often comes from processing this information quickly enough to detect anomalies, trigger alerts and make real-time decisions.
- **ML Pipelines:** Training models require large historical datasets, while feature engineering, experimentation, and model versioning generate additional storage and processing requirements. In many organizations, ML has become one of the largest consumers of big data infrastructure.

Although all these workloads look very different, they all share a common characteristic. They generate data continuously, at scale, and often require distributed storage and processing systems to extract meaningful value from that data.

Batch Vs. Stream Processing

Collecting and storing data creates potential value, but organisations benefit only when their data can be processed into actionable insights. The question becomes, how quickly do you need those insights. That leads us to 2 fundamental processing models used in big data systems, batch processing and stream processing.



Batch processing works on accumulated datasets. Data is collected over a period of time, and then processed as a large job. This approach is highly efficient for handling massive amounts of data, making it ideal for ETL pipelines, business reporting, analytics, and large-scale aggregation. The trade-off here is latency. You might wait minutes, hours, or even a full day before results are available.

Stream processing takes the opposite approach. Instead of waiting for data to accumulate, it processes events continuously as they arrive. This enables near-real-time insight, which is critical for use cases like fraud detection, operational monitoring, recommendation engines, and IoT analytics. The challenge here is increased architectural complexity, because the system must handle continuous data flow with very low latency.

An easy way to think about this difference is that batch processing answers questions about what happened, while stream processing helps to react to what is happening right now. In practice, most large-scale systems use both. Initial analysis and large transformations often run in batch, while time-sensitive decisions rely on stream processing.

As architects, the key decision isn't choosing one over the other. It is understanding the latency requirements of the business, and then selecting the right processing model for the current job.

Summary & Key Takeaways

- Big Data is defined by the 5 V's.
- Traditional storage struggles with scale and velocity.
- Distributed systems like HDFS, S3, and Delta Lake address these challenges.
- Big Data workloads span multiple industries and domains.
- Batch and Stream processing are key paradigms for handling big data.
- What's next:
 - Choosing the Right Storage Solution & Final Recap of storage section

Interview Questions

- What are the 5 V's of Big Data? Why are they important?
- Why do traditional databases struggle with Big Data workloads?
- Compare HDFS and S3. In what scenarios would you choose one over the other?
- What types of workloads qualify as Big Data problems?
- What is the difference between batch and stream processing?
- What is Delta Lake and how does it improve upon traditional data lakes?
- How would you design a system to process terabytes of log data daily?
- What storage and processing frameworks would you use and why?